

The background of the slide is a dark blue gradient. On the left, there are several overlapping, semi-transparent browser window icons showing various web page layouts. On the right, there is a semi-transparent globe with a network of glowing blue lines and dots connecting different points, symbolizing global connectivity and data flow. The title 'Web Scrapping' is centered in the middle of the slide, with 'Web' in white and 'Scrapping' in a bright blue color. Below the title, the author's name 'Raúl Salas' is written in a smaller, white font.

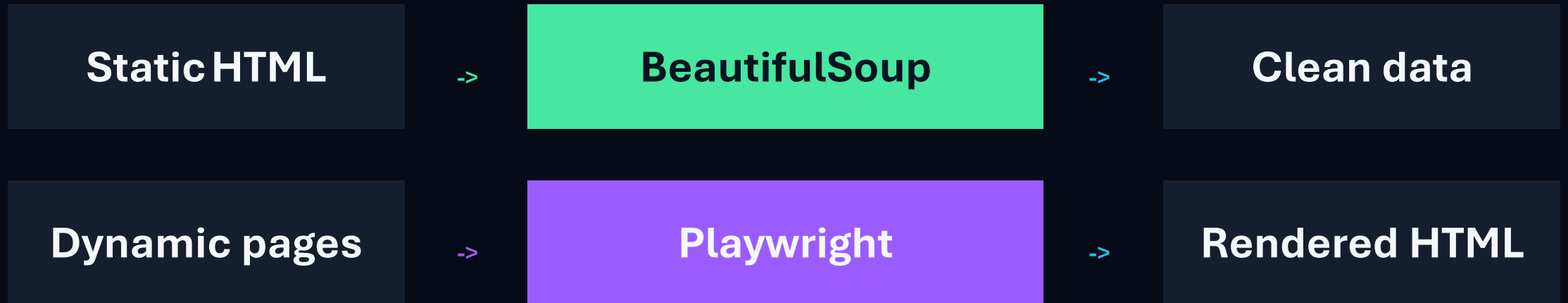
Web Scrapping

Raúl Salas

Web Scraping Tools

BeautifulSoup for static HTML. Playwright for dynamic pages.

Use the lightest tool that can actually see the data.



In practice: Playwright can load the page; BeautifulSoup can extract from the resulting HTML.

BeautifulSoup

Best when the data is already in the HTML.

Simple. Lightweight. Fast.

Low setup: no browser needed

Readable selectors: target fields quickly

Easy cleanup: HTML to structured data

Less overhead: faster for static pages

```
1 html = requests.get(url).text
2 soup = BeautifulSoup(html, "html.parser")
3 rows = soup.select("table tr")
```

Ideal for
tables, listings, product pages,
and saved HTML

Main advantage: it stays simple when the website is simple.

Playwright

Best when the page needs a real browser.

JavaScript. Clicks. Waiting.

Dynamic content: JavaScript-rendered pages

Real interactions: clicks, scrolls, logins

Auto-waiting: fewer fragile sleeps (Playwright Stealth)

Debugging: screenshots and traces

```
1 page.goto(url)
2 page.get_by_text("Search").click()
3 cards =
  page.locator(".card").all_text_contents()
```

Ideal for
dynamic sites, infinite scroll,
forms, and logins

Main advantage: it behaves like a user, so it can reach data that static parsing cannot.